

# Mini-Transformer from Scratch

Scaler School of Technology | Deep Learning I | Week 8 — Session 5/5 | Post-Lecture Review & Practice



## Key Takeaways

### 1 Transformers replace recurrence with parallelism

Instead of processing tokens sequentially, self-attention computes  $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V$  for all positions simultaneously. Every token attends to every other token in  $O(n^2d)$  operations — quadratic in sequence length but constant in depth, enabling massive parallelism on GPU.

### 2 Multi-head attention captures diverse relationships

$h$  parallel heads each project into  $d_k = d_{\text{model}}/h$  dimensional subspaces with separate  $W_j^Q, W_j^K, W_j^V$ . Each head can learn a different attention pattern — one tracking syntactic dependencies, another semantic co-reference, another positional proximity. Outputs are concatenated and projected back:  $\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$ .

### 3 Positional encoding is critical for sequence awareness

Self-attention is **permutation-invariant**: without positional information, “dog bites man” and “man bites dog” are identical. Sinusoidal PE adds unique position fingerprints:  $PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$ ,  $PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$ . Different frequencies encode position at multiple scales.

## Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

- Q1. Why do we divide attention scores by  $\sqrt{d_k}$  before applying softmax? What problem does large  $d_k$  cause without this scaling?
- Q2. The decoder uses both *masked self-attention* and *cross-attention*. What is the specific purpose of each, and what does each attend to?
- Q3. Walk through the data flow when the transformer processes input “1 2 3” to generate the first output token “one”. Name each component and its transformation.
- Q4. You remove positional encoding from your trained model. Which input would fail first: single-digit (1–9), two-digit (10–99), or three-digit numbers (100–999)? Explain.
- Q5. Estimate the total parameter count of a transformer with  $d_{\text{model}} = 128$ , 4 heads, 2 encoder layers, 2 decoder layers,  $d_{\text{ff}} = 256$ , and source/target vocab sizes of 20 each. Show your reasoning.

## Coding Exercises

Work through these in a Colab notebook with GPU enabled.

### Exercise 1: Implement Scaled Dot-Product Attention from Scratch

Write a function `scaled_dot_product_attention(Q, K, V, mask=None)` that: (a) computes scores  $QK^\top / \sqrt{d_k}$ , (b) applies mask by setting masked positions to  $-\infty$  if provided, (c) applies softmax, (d) returns output = weights  $\times V$  and the weight matrix. Test with Q,K,V of shape (2, 5, 16). Verify output shape is (2, 5, 16) and each row of the weight matrix sums to 1. Visualise the weight matrix as a heatmap. *Expected output*: shape verification + 2×5×5 attention heatmap.

### Exercise 2: Positional Encoding Analysis

Generate sinusoidal PE for 50 positions with  $d_{\text{model}} = 128$ . (a) Plot the PE matrix as a heatmap (rows = positions, columns = dimensions). (b) Compute cosine similarity between position 0 and all other positions — plot the decay curve. (c) Verify: shuffle the PE rows randomly and check that the matrix looks qualitatively different from the original. *Expected output:* PE heatmap + cosine similarity decay plot showing nearby positions are more similar.

### Exercise 3: Visualise Multi-Head Attention Patterns

Train the full mini-transformer on numbers 1–800. After training, process input “1 2 3” and extract attention weights from ALL 4 heads in the *first encoder layer*. Create a  $2 \times 2$  grid of attention heatmaps (one per head), with source tokens on both axes. Describe in 1 sentence what each head appears to attend to. *Expected output:* 4-panel attention visualisation showing diverse attention patterns across heads.

## Challenge Problem

**Adapt the Transformer for Roman Numeral Conversion** (“XIV” → “fourteen”):

1. Generate dataset: Roman numerals for all integers 1–3999 (use Python’s `int_to_roman` function)
2. Build a character-level vocabulary for Roman numeral input (I, V, X, L, C, D, M + special tokens)
3. Reuse the *exact same* transformer architecture — no architectural changes needed
4. Train for 50 epochs on numbers 1–3000, test on 3001–3999
5. Report exact-match accuracy on the test set

*Bonus:* Visualise cross-attention to see which Roman characters the model attends to when generating each output word (e.g., does it attend to “X” when generating “ten”?).

## Further Reading

- **Original Paper:** Vaswani et al. (2017), “*Attention Is All You Need*” — focus on Sections 3.1–3.3 (self-attention, multi-head, positional encoding).
- **Formal Reference:** Phuong & Hutter (2022), “*Formal Algorithms for Transformers*” — rigorous pseudocode for every component; use as an implementation reference.
- **GPT:** Radford et al. (2018), “*Improving Language Understanding by Generative Pre-Training*” — shows this exact architecture scales to language understanding at 117M params.
- **BERT:** Devlin et al. (2019), “*BERT: Pre-training of Deep Bidirectional Transformers*” — encoder-only variant; demonstrates the power of pre-training + fine-tuning.

## Online Resources & Tools

- **Annotated Transformer** — Harvard NLP impl.
- **BertViz** — attention head visualisation
- **3Blue1Brown** — visual attention explainer
- **nanoGPT** — Karpathy’s minimal GPT
- **PyTorch nn.Transformer** — official docs
- **HuggingFace transformers** — production models

## Fill the Gaps — Prerequisite Refreshers

*Felt lost on a specific concept? These resources will help you catch up.*

- **Confused about residual connections (Add & Norm)?** Revisit Week 4 ResNet lecture — residuals  $x + F(x)$  allow gradients to bypass layers entirely, critical at depths  $>10$  layers.
- **LayerNorm vs BatchNorm?** LayerNorm normalises across feature dimensions *per token*:  $\hat{x} = (x - \mu) / \sigma \cdot \gamma + \beta$  where  $\mu, \sigma$  are computed per token. No batch-size dependency, works for any sequence length.
- **Forgot causal masking?** The upper-triangular mask fills future positions with  $-\infty$  before softmax, ensuring  $\alpha_{ij} = 0$  for  $j > i$ . Without it, the decoder can “cheat” by looking at future output tokens.
- **Unsure about teacher forcing in transformers?** Same concept as Session 4 — during training, feed the entire shifted target sequence in parallel (not token-by-token), masking future positions. At inference, generate autoregressively.

## Answers — Test Your Understanding

- A1.** Large  $d_k$  causes dot products  $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$  to grow with  $d_k$  (sum of  $d_k$  terms). Large logits push softmax into saturation where gradients become near-zero (one position gets attention weight  $\approx 1$ , all others  $\approx 0$ ). Dividing by  $\sqrt{d_k}$  keeps the variance of the dot product  $\approx 1$  under unit-variance Q, K assumptions.
- A2.** *Masked self-attention*: allows each output position to attend to all *previous* output positions — the decoder uses its own previously generated tokens to build context. The causal mask prevents attending to future tokens. *Cross-attention*: allows each output position to attend to all encoder outputs (the full source) — this is how source information is incorporated into each decoding step.
- A3.** Source digits  $\rightarrow$  token embeddings (shape [1,3,128]) + PE  $\rightarrow$  2 encoder layers (each: self-attn + FFN + Add&Norm)  $\rightarrow$  encoder output [1,3,128]. Decoder starts with <SOS> + PE  $\rightarrow$  masked self-attn  $\rightarrow$  cross-attn (over encoder output)  $\rightarrow$  FFN  $\rightarrow$  linear layer [1,1,vocab]  $\rightarrow$  softmax  $\rightarrow$  argmax “one”.
- A4.** Three-digit numbers (100–999) fail first. For single/two-digit inputs, the model could potentially identify digit identity from the embedding alone. Three-digit numbers require distinguishing hundreds, tens, and units positions — “1 2 3” (one hundred twenty-three) vs “3 2 1” (three hundred twenty-one) are completely different outputs but their embeddings without PE are permutations of each other.
- A5.** Per encoder layer: MHA =  $4 \times (3 \times 32 \times 128 + 128^2) \approx 32\text{K}$ ; FFN =  $128 \times 256 + 256 \times 128 \approx 65\text{K}$ ; total per encoder layer  $\approx 97\text{K}$ . Per decoder layer:  $2 \times \text{MHA} + \text{FFN} \approx 129\text{K}$ . 2 encoder + 2 decoder  $\approx 452\text{K}$ . Embeddings:  $2 \times 20 \times 128 = 5\text{K}$ . Total  $\approx 460\text{K}$  parameters.

## Deep Dive: Self-Attention — From Intuition to Equations

The **database lookup analogy**: each token is a query ( $Q$ ) searching a database of keys ( $K$ ) to retrieve values ( $V$ ). High query-key dot product  $\Rightarrow$  high attention weight  $\Rightarrow$  more of that value is included.

**Full computation** for a sequence of 3 tokens with  $d_{\text{model}} = 4$ ,  $d_k = 4$ :

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \quad (\text{linear projections})$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

The attention matrix  $A = \text{softmax}(QK^\top/\sqrt{d_k}) \in \mathbb{R}^{n \times n}$  has entry  $A_{ij}$  = how much token  $i$  attends to token  $j$ . Row  $i$  sums to 1.

## Deep Dive: Layer Normalisation

LayerNorm normalises each token's representation across the feature dimension:

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta$$

where  $\mu = \frac{1}{d} \sum_i x_i$ ,  $\sigma^2 = \frac{1}{d} \sum_i (x_i - \mu)^2$ , and  $\gamma, \beta$  are learnable scale/shift.

**Why not BatchNorm?** BatchNorm normalises across the *batch* dimension per feature — requiring fixed batch size and not working for variable-length sequences. LayerNorm normalises per *token* across features — batch-size independent, works for any sequence length.

The full transformer sub-layer:  $\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$  where sublayer is either self-attention or FFN.

## Deep Dive: Encoder vs Decoder

| Feature                                                                              | Encoder                   | Decoder                                 |
|--------------------------------------------------------------------------------------|---------------------------|-----------------------------------------|
| Self-attention mask                                                                  | None (bidirectional)      | Causal mask (upper-triangle $-\infty$ ) |
| Cross-attention                                                                      | No                        | Yes (attends to encoder output)         |
| Processing order                                                                     | All positions in parallel | Autoregressive at inference             |
| Example models                                                                       | BERT, RoBERTa, ViT        | GPT-1/2/3/4                             |
| Use case                                                                             | Understanding/encoding    | Generation                              |
| <i>Encoder-decoder together</i> : T5, original Transformer, BART — for seq2seq tasks |                           |                                         |

## Ablation Experiments

Each experiment isolates one component. Form a hypothesis before running:

1. **No positional encoding:** Remove PE entirely. Test accuracy on 1-digit, 2-digit, and 3-digit numbers separately. Which class degrades most? Does the model find alternative strategies?
2. **Single head vs. 4 heads:** Replace multi-head attention with single-head attention ( $d_k = d_{\text{model}}$ ). Compare accuracy and attention pattern diversity.
3. **Depth ablation:** Train with 1, 2, 3, 4 encoder/decoder layers. Plot test accuracy vs. number of layers. At what depth does overfitting begin on this small dataset?
4. **Model dimension:** Try  $d_{\text{model}} \in \{32, 64, 128, 256\}$ . Plot accuracy vs. parameter count. Find the minimum capacity needed for  $> 95\%$  exact match accuracy.
5. **No LR warmup:** Train with constant learning rate instead of warmup + decay. Does training instability occur? Compare final accuracy and loss curves.

## From Mini-Transformer to GPT

The architecture you built *is* GPT — a decoder-only transformer, trained autoregressively. The **only differences** between your model and GPT-3:

| Dimension          | Your Model            | GPT-3              |
|--------------------|-----------------------|--------------------|
| Parameters         | ~460K                 | 175B               |
| Layers             | 2 encoder + 2 decoder | 96 decoder-only    |
| $d_{\text{model}}$ | 128                   | 12,288             |
| Attention heads    | 4                     | 96                 |
| Training tokens    | <1K                   | 300B               |
| Training compute   | Minutes (Colab)       | ~\$4M in GPU hours |

The math, the training loop, and the autoregressive generation procedure are identical. Scaling laws (Kaplan et al., 2020) show performance improves smoothly and predictably with scale — there are no qualitative architectural changes between your model and GPT-4.

## The Course Arc: Perceptron to Transformer

| Week | Model               | Key Innovation                            | Limitation Addressed   |
|------|---------------------|-------------------------------------------|------------------------|
| 1    | Perceptron          | Learned threshold                         | Hard-coded rules       |
| 2–3  | MLP + Backprop      | Hidden layers + gradient descent          | XOR / non-linearity    |
| 4    | CNN                 | Parameter sharing, local receptive fields | Spatial invariance     |
| 5    | Transfer Learning   | Pre-trained features                      | Data scarcity          |
| 6    | LSTM                | Gating, cell state                        | Vanishing gradients    |
| 6–7  | Seq2Seq + Attention | Soft alignment over all positions         | Information bottleneck |
| 7–8  | Transformer         | Parallel self-attention                   | Sequential computation |

## Transformer Hyperparameter Guide

| Parameter           | Typical Range                         | Effect                      | How to Choose                                           |
|---------------------|---------------------------------------|-----------------------------|---------------------------------------------------------|
| $d_{\text{model}}$  | 64–512 (small); 768–4096 (large)      | Representation capacity     | Must be divisible by $n_{\text{heads}}$                 |
| $n_{\text{heads}}$  | 2–16                                  | Multi-perspective attention | $d_{\text{model}}/n_{\text{heads}}$ should be $\geq 16$ |
| $n_{\text{layers}}$ | 2–6 (small); 12–96 (large)            | Depth / compositionality    | Increase until overfitting                              |
| $d_{\text{ff}}$     | Typically $4 \times d_{\text{model}}$ | FFN bottleneck capacity     | $4\times$ is standard                                   |
| Dropout             | 0.1–0.3                               | Regularisation              | 0.1 for small models                                    |
| LR (peak)           | $10^{-4}$ – $10^{-3}$                 | Convergence speed           | Use warmup + decay                                      |
| Warmup steps        | 5–10% of total steps                  | Training stability          | Required for Adam                                       |
| Max sequence length | Task-dependent                        | Context window              | Longer $\Rightarrow$ more memory ( $O(n^2)$ )           |

## What to Learn Next: A Roadmap

### Short term (1–3 months):

- Fine-tune a pre-trained BERT/GPT-2 on a classification task using HuggingFace transformers
- Build a simple RAG (Retrieval-Augmented Generation) system using LangChain
- Complete fast.ai Part 1 or Andrej Karpathy's "Zero to Hero" YouTube series

### Medium term (3–6 months):

- Stanford CS224N (Natural Language Processing with Deep Learning) — free online
- Implement nanoGPT from scratch following Karpathy's lecture
- Learn LoRA and PEFT — fine-tune LLMs efficiently without full parameter updates

### 10 Papers Every DL Practitioner Should Read:

- Attention Is All You Need (2017)
- BERT (2019)
- GPT-3 (2020)
- Vision Transformer (ViT, 2021)
- CLIP (2021)
- InstructGPT (2022)
- LoRA (2022)
- Stable Diffusion (2022)
- Mamba (2023)
- Mistral 7B (2023)

## Debugging Checklist

| Symptom                       | Likely Cause                      | Fix                                                    |
|-------------------------------|-----------------------------------|--------------------------------------------------------|
| Loss NaN in first epoch       | LR too high, no warmup            | Add warmup; reduce peak LR by $10\times$               |
| Attention weights all uniform | Dot products too small            | Check $d_k$ scaling ( $\sqrt{d_k}$ divisor)            |
| Decoder generates same token  | Causal mask missing               | Verify upper-triangle mask is applied in decoder       |
| Accuracy plateau at 50%       | Model too small or data too noisy | Increase $d_{\text{model}}$ ; check vocabulary mapping |
| Training much slower than RNN | Batch size too small              | Increase batch size (Transformers need it)             |

Hands-on notebook: `Session_5_Transformer_From_Scratch.ipynb`

Instructor: Priyansh Saxena